

WorkWebs Audio Mixing Ecosystem CUE SYSTEM Documentation

By Larry T. Jost

Version 20260916a

[WorkWebs.com](https://github.com/ltjost/WorkWebs-Audio-Mixing-Ecosystem-Tools) <https://github.com/ltjost/WorkWebs-Audio-Mixing-Ecosystem-Tools>

Support@WorkWebs.com

<https://www.facebook.com/profile.php?id=61557113121231>

IMPORTANT:

The tools I have written are available from [WorkWebs.com](https://www.workwebs.com). External components are available as shown below.

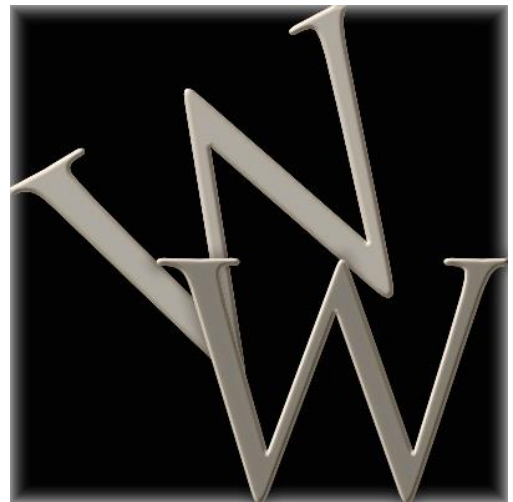
Use REAPER 7.25 or later.

There is no warrantee expressed or implied. Use this at your own risk. I am not responsible for any errors, omissions, or defects in this software solution. Be sure to test everything you use and verify its operation is to your satisfaction.

You may rework any of this as long as you do not require payment for your derived product and as long as you reference WorkWebs.com as the source for the original software.

If you would like to send a Thank You gift you may do so from this link or QR code:

<https://www.paypal.com/ncp/payment/7W434P69MWXA8>



Contents

Important Notes:	8
Changes made in this release	8
Cue System	8
Overview	8
Setting up the BUILD Script	8
Type	9
myFileExtension	9
The RUN script	10
Edit the Cue list in the RUN script	11
PASSTHROUGH	12
CUE FILE Content for PASSTHROUGH Commands	12
WING	12
BUILD Script Modifications for WING Cues	12
TrackVar Commands	12
SEND IDENTIFIERS	13
Colors	14
Connection Commands	15
TAG Commands	15
CUES for the Behringer WING	15
CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS WING	16
createfile	17
CUSTOMStartCue	17
CUSTOMEndCue	17
TRACKMUTE	17
TRACKLED	17
TRACKNAME	17
TRACKCOLOR	17
TRACKICON	18

TRACKPAN	18
TRACKPOSTINS	18
TRACK_GATE_ON	18
TRACK_GATE_OFF	18
TRACK_EQ_ON	18
TRACK_EQ_OFF	18
TRACK_DYN_ON.....	18
TRACK_DYN_OFF	18
TRACK_PREINS_ON	19
TRACK_PREINS_OFF	19
TRACK_POSTINS_ON	19
TRACK_POSTINS_OFF	19
TRACKPROC	19
TRACKTAG	19
TRACKTAGMUTE.....	19
TRACKGROUPIN.....	19
TRACKSEND	20
SENDand_UNMUTE.....	20
SENDand_MUTE.....	20
USB_PLAY	20
USB_PLAYFILE.....	20
USB_RECORD	20
LIVE_RECORDER	20
LOADSCENE	20
NAVSCENE	20
AUTOMIXXON.....	21
AUTOMIXXOFF	21
AUTOMIXYON.....	21
AUTOMIXYOFF.....	21

CUSTOM_a_DCA_ONMUTED.....	22
CUSTOM_a_DCA_ONUNMUTED.....	22
CUSTOM_a_DCA_OFF	22
Custom_a_CHAN_ON	23
Custom_a_CHAN_OFF	23
SETLATESTDCA.....	24
SETTRACKTOLATESTDCA	24
# 	26
X32 M32	26
CUES for X32/M32.....	26
Detail Using an Example for X32/M32 BUILD Script Modifications	27
Channels.....	27
BUS IDENTIFIERS	27
SEND IDENTIFIERS	27
DCA Assign Identifiers	28
DCA Identifiers	29
Route Out Identifiers and Route Source Out	29
Route In Identifiers and Route Source In	31
Colors	32
Card Commands.....	32
USB Stick Commands	33
CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS X32	33
createfile	33
CUSTOMStartCue	33
CUSTOMEndCue.....	34
CHAN	34
CHANNAME.....	34
CHANCOLOR.....	34
CHANSEND	34
CHANDCA	34

DCA.....	34
DCANAME	35
DCACOLOR	35
BUS.....	35
BUSNAME.....	35
BUSCOLOR.....	35
MAINST	35
MAINSTNAME	35
MAINSTCOLOR	35
MAINM.....	36
MAINMNAME	36
MAINMCOLOR.....	36
SENDand_UNMUTE.....	36
SENDand_MUTE.....	36
ROUTE_OUT	36
ROUTE_IN.....	36
USB_RECORDER	36
LIVE_RECORDER	36
SAVESCENE	37
LOADSCENE	37
SAVESNIPPET	37
LOADSNIPPET	37
# 	37
REAPER.....	38
Sample for REAPER BUILD SCRIPT MODIFICATIONS	38
Build Script Edits	38
Track (Channel) Naming	38
Send Identifiers (tracks to bus tracks sending, monitor sending, etc. based on your configuration).....	39
Plugin Adjust identifiers (REAPER only, experimental)	40

Sample of Cue File Content (REAPER syntax) for reference	40
CUES for REAPER	41
One Example Cue File for reference	42
REAPER Cue Content-Command Syntax and Definitions	45
createfile	45
CUSTOMStartCue	45
CUSTOMEndCue.....	45
SENDand_UNMUTE.....	46
SENDand_MUTE.....	46
SENDand_ezMUTE	46
SENDand_UNMUTE_NAME	46
SENDand_MUTE_NAME	46
TRACKPAN	46
SELECT	46
UNSELECT	47
UNMUTE	47
MUTE.....	47
MARKERName	47
LASTMARKERName	47
GOTOMarker	47
RECORD	47
PLAY.....	47
STOP	48
PAUSE	48
FXBYPASS.....	48
FXACTIVE	48
TRACKNAME.....	48
TRACKVOLUME	48
ACTIONi	48
ACTIONS	48

ARMON	49
ARMOFF	49
AUTOMIXON.....	49
AUTOMIXOFF	49
PLUGIN_ADJ	49
FXOPENUI	49
FXCLOSEUI	49
# 	49
AutoHotKey Script.....	50
Bitfocus Companion and Vireo Listener	50
Links	51

Important Notes:

- See the readme file for additional information on all the files in the package.

Changes made in this release

- **WING now includes the ability to change track color when muting.**
- **Cue files now include a target IP and Port for each OSC command allowing multiple devices to be impacted by a single cue file (1 or more WINGS + 1 or more X32s + 1 or more REAPERs + 1 or more of anything using OSC via passthrough commands.**

Cue System

Overview

The CUE system uses Cue files to hold cues. The Cues are text files you write using commands custom designed in this software and can be sent to any device. Each line of the cue file specified a target device . Passthrough raw OSC commands can also be included to send to software you are using even if this system does not have special handling for the desired device (such as MultiPlay for playing audio files). The custom defined commands interpreted in this tool are for Behring WING or X32 or REAPER.

The BUILD CMD Script is run to process the CUES and build OSC commands with identifiers for each target device.

The RUN CMD script is used to run the cues built by the BUILD script in real time.

The system uses SENDOSC to send commands. This is available at:

<https://github.com/yoggy/sendosc>

Remote control of the cue execution can be achieved with Bitfocus Companion with Vicreo Listener, a configuration is provided with this package.

You are also able to skip the build process and just make your own output files and they will still work with the RUN script. This allows you to write OSC and send it to any device you want, either in a sequence or by name as desired without the need to use the BUILD process.

ALL CUES, BUILD, and RUN scripts and the SENDOSC executable and associated installation files, if any, should all be placed the SAME DIRECTORY for this system to operate.

Setting up the BUILD Script

Edit the BUILD script to include your definitions. Use the sample provided as a starting point.

Type

Each device you will send commands to needs to be identified by name. In the sample file I set up 2 WINGs and 1 REAPER and 1 X32 and 1 PASSTHROUGH device for my cues.

Here WING1 is the identifier I use in the cues for commands to be sent to a WING I have named WING1. Similarly the name WING2 is a WING mixer, REAPER1 is a REAPER instance, X1 is an X32, and I have one name for PASSTHROUGH named PASSTHROUGH.

```
set Type.WING1=WING  
set Delay.WING1=0
```

```
set Type.WING2=WING  
set Delay.WING2=0
```

```
set Type.REAPER1=REAPER  
set Delay.REAPER1=0
```

```
set Type.X1=X32  
set Delay.X1=0
```

```
set Type.PASSTHROUGH=PASSTHROUGH  
set Delay.PASSTHROUGH=0
```

Note that “Delay” is no longer used but still included because I have not cleaned it out yet so it is an attribute for each device Name I specified.

PASSTHROUGH is not a specific device but is needed so the code in Build works properly, so just include this as shown.

These Names are used in CUES and are assigned IP addresses and Ports in the RUN script. Because of this name use in cues you do not need to rebuild cues if IP addresses or Ports change, just change the attributes in the Run script and you are set.

myFileExtension

Next set your file extension in the Build script. For example:

```
set myFileExtension=cuex
```

Files with the specified extension in the cue folder will be processed as cues.

The RUN script

Cues are created with an ordered sequence. You do not need to process them in that sequence though if you do not want to. You can simply enter the “cuename” (getready or go in this example) to run the desired cue. To run them in sequence, you would just hit enter to go to the next cue. (You can back-up by entering “p”.)

You can use a Cue File in more than one cue, but each cue needs a unique cuename value.

The cuedescr value is displayed in the command prompt window when the cue is run. You can wait a while before running a cue. The number of seconds to wait is set in the afterpause variable for the cue. If it is omitted then 0 is used.

If you provide a cue name assignment to the follow variable then the specified cue will run after the current cue is processed without you taking any further action. This is handy if you want to have a cue impact multiple devices because you can trigger one cue and the additional device cues will run automatically. Remember, an individual cue file is destined to a single device by a cue based on the prefix letter.

The last thing in each cue definition is an indication of how to process the cue. The choices are “call” or “start”. If you specify “call” then the RUN script will process the cue and wait until it completes before allowing further action. If you specify “start” then an additional command prompt window is opened and runs the cue and the RUN script is ready for additional processing as soon as the cue is started.

There are lots of design choices to make when writing cues. Keep in mind that a lot of commands are available and can result in many OSC commands. If you anticipate running cues out of sequence, then each cue needs to get the entire system in a suitable state. If this involves setting 30 send values on each of 30 tracks, 30 track mutes, 30 track namings, and other things the cue may take longer than desired to execute.

After cues are built, the BUILD script is set up and the RUN script is set up, it is time to run the BUILD script. Open a command prompt window and change to the directory of the script and cues. The run your customized BUILD script. It will build an output cue file for each input cue file associated with project/file extension. If there are errors you will be notified. Once this completes, either open another command prompt window so you can get back to build quickly, or use the same window and execute the customized RUN script. Then follow the instructions to run your cues.

The full set of available input cue commands are described later in this document and are provided for REAPER or for X32/M32, PASSTHROUGH or for the Behringer WING. The commands available to each device type differ so your cues need to use the appropriate BUILD file and cue content. BUILD templates are provided for REAPER and X32/M32 and the WING. Only 1 RUN script is required to process any of them though.

The RUN script is to be modified to include the IP and Port for each target device named in the BUILD script.

For example, with 2 WINGS and 1 X32 and 1 MultiPlay (Passthrough cues) instance per the example indicated earlier for the BUILD script:

```
set IP.WING1=192.168.0.11
set Port.WING1=2223
```

```
set IP.WING2=192.168.0.12
set Port.WING2=2223
```

```
set IP.REAPER1=192.168.0.107
set Port.REAPER1=8007
```

```
set IP.MULTIPLAY1=192.168.0.107
set Port.MULTIPLAY1=8888
```

Ignore references I the script about using a single letter code. That was the old way of doing this but the script was not edited to remove these references.

Edit the Cue list in the RUN script

As described above, each cue needs to be defined with a sequence number and some processing information. Here is an example:

```
:: FOR EACH CUE
```

```
set /a cuecount+=1
set cue[%cuecount%].curname=10
set cue[%cuecount%].file=10
set cue[%cuecount%].cuedescr=mute them all and all out of DCAs
set cue[%cuecount%].afterpause=
set cue[%cuecount%].follow=
set cue[%cuecount%].execute=call
```

```
set /a cuecount+=1
set cue[%cuecount%].curname=20
set cue[%cuecount%].file=20
set cue[%cuecount%].cuedescr=all in their dcas BUT DCAs Muted
set cue[%cuecount%].afterpause=
set cue[%cuecount%].follow=
set cue[%cuecount%].execute=call
```

Note that you can run multiple RUN scripts on one or more networked computers. You may want to do this to aid in operational effectiveness or you may want to run sequential cues plus have non-sequential “commands” as cues in another setting. This lets you use utility cues at any time while not interrupting sequential flows. Using AutoHotKey (or similarly Bitfocus Companion) you can set up keyboard keys as single press commands for any of the RUN script. This is a great way to build a custom keyboard to process custom commands that are using OSC control!

PASSTHROUGH

CUE FILE Content for PASSTHROUGH Commands

A passthrough command is added to a cue file as follows. This example passes /cue/10/go to the device defined in the RUN script as MULTIPLAY1 using the RUN script designated IP and Port for MULTIPLAY1

```
PASSTHROUGH|MULTIPLAY1^^/cue/10/go
```

WING

BUILD Script Modifications for WING Cues

TrackVar Commands

Each “track” on the WING needs a name and some attributes. This is done by setting attributes for the myTrackVar.

Example for Channel 1:

```
set myTrackVar.w1cCaroline=1
set myTrackVar.w1cCaroline.type=ch
set myTrackVar.w1cCaroline.color.on=%myWingColor.PINK
set myTrackVar.w1cCaroline.color.off=%myWingColor.GRAY
```

In Cues Channel 1 is now defined as “w1cCaroline” It is set as a “ch” or channel and includes optional attributes of an On (unmute) and Off (mute) color setting to establish with mute and unmute commands.

Note that a channel can have 1 or more names, each with its own attributes.

Type values can be: ch, aux, mtX, bus, dca, main for general use for channels, auxs, matrixes, busses, dcas and main track types.

Type can be of the form io/in/LCL (where LCL is an example) to impact an io/in or io/out/LCL (where LCL is an example) or any other OSC desired.

Note that all types do not support all commands, see the manual of OSC commands.

Example for naming the Local 1 In source:

```
set myTrackVar.iLCL1=1  
set myTrackVar.iLCL1.type=io/in/LCL
```

Note that all types do not support all commands, see the manual of OSC commands for the WING that is available online.

An example for a DCA:

```
set myTrackVar.w1dENS4=5  
set myTrackVar.w1dENS4.type=dca
```

The name w1dENS4 is used in cues and is DCA 5 and of course the type is set to dca.

Also set TAG variable, see below, attributes for the TAGS you will use on the WING.

Here are examples for 5 DCAs:

```
set myTAGVar.w1dcaOTHER="#D1"  
set myTAGVar.w1dcaENS1="#D2"  
set myTAGVar.w1dcaENS2="#D3"  
set myTAGVar.w1dcaENS3="#D4"  
set myTAGVar.w1dcaENS4="#D5"
```

SEND IDENTIFIERS

In the SENDS designations note the following meaning of the SEND strings:

1 sets the send in this position to ON, or Unmuted

0 sets the send in this position to OFF, or Muted

X leaves the send alone

* terminates the processing of the send string

- is ignored and does not even count as a position

Sample from a script:

```
:: SENDS NEVER CHANGE  
set mySend.sdonothing=*
```

```
set mySend.svoxsetup=111-000-00-00-111111*  
set mySend.sinstsetup=000-111-11-00-111111*
```

The names specified here are used in the cue files.

Each 1 or 0 specified what to do with the send at that sequence position.

For example solo is:

*1000**

Position 1 is a 1 so BUS 1 Send would be set ON for the specified channel when this SEND is used in a command.

The other 0's in position 2, 3,4 mean BUS 2, 3, 4 Sends for the referenced Channel will be set to OFF.

– is a spacer and does not have any meaning or cause the position to increment, it just helps make things readable in the code.

* Ends the string of entries

Colors

Colors for WING scribble strips are passed in OSC as numbers, but designated in CUEs as names. A default set of names is provided in the BUILD script, but you can change these.

These lines define the color choices:

```
set myColor.LIGHTBLUE=1  
set myColor.MEDIUMBLUE=2  
set myColor.DARKBLUE=3  
set myColor.TURQUOISE=4  
set myColor.GREEN=5  
set myColor.OLIVEGREEN=6  
set myColor.YELLOW=7  
set myColor.ORANGE=8  
set myColor.RED=9  
set myColor.CORAL=10  
set myColor.PINK=11  
set myColor.MAUVE=12  
set myColor.GNi=10  
set myColor.YEi=11  
set myColor.BLi=12  
set MyColor.MGi=13  
set myColor.CYi=14  
set myColor.WHi=15
```

In CUEs you refer to these by GREEN, YELLOW, etc.

Connection Commands

The WING channels need to have connections set in place to indicate the source for the channel. This is done by a Group designation and a number designation within the group. See the WING OSC manual for details, but for example, a Channel connection can be from any of these groups: OFF, LCL, AUX, A, B, C, SC, USB, CRD, MOD, PLAY, AES, USR, OSC, BUS, MAIN, MTX with a number from 1-64. Likewise for an AUX.

In the BUILD script you may say:

```
set myCONNVar.micSEN1.type=B  
set myCONNVar.micSEN1.num=1
```

to indicate that a reference to micSEN1 refers to AES50 B 1.

In a CUE you could use:

```
TRACKGROUPIN|cJenny|micSEN1|
```

to indicate that the channel named cJenny is to be connected to whatever micSEN1 indicated, namely AES50 B 1.

TAG Commands

TAGs in the WING are used for a variety of uses, but in the CUEs they are set up specifically for DCA handling. This is done with the myTAGVar variables such as:

```
set myTAGVar.dca1="#D1"  
set myTAGVar.dcaJO="#D1"
```

Multiple DCA tag can be used, just separate them with commas. DCA Tags are #D1...#D16. With the above in the build file, by cues can reference dca1 or dcaJo and either will refer to that #D1 with is the tag for the first DCA.

CUES for the Behringer WING

Each Cue file that you build contains commands that the BUILD script will read and create an expanded cue file for. Those cue files will be used by the RUN script later to run your cues.

Next is the detail on the commands that you place in a CUE file. Note that if you build these files in sequence that you limit the amount of adjustment you need to make in them and it really is pretty fast.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can used Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

```
reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

For the Current User and setting the sampleext extension to be recognized as text:

```
reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

Note that Notepad++ is freely available and can be used to make mass changes to multiple files at once if needed to adjust your cue file.

Commands in the cue files are executed in the order received, so be sure you consider that when placing the content. Things that must be done quickly should be first.

Because cues can be run out of sequence, it is good practice to set everything that can change in each cue or setup special setup cues and process cues so that, for example, DCA assignments can be made where there is time and then use simpler Mute/Unmute cues within that DCA assignment scope.

Note that cue names and cue sequence are not the same. The sequence of cue file execution is specified in the RUN script.

Next below are the definitions of the commands that can be placed in the cue files. This will help you see what you can designate.

To build the Cue files for the RUN script to process, just run the BUILD script from within the directory with the other items and it will build all the cue expanded files that the RUN script will process.

CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS WING

NOTE: Values designated as “myXxx value” are defined in the WING BUILD script and described above.

BE SURE to see the details and sample under SETLATESTDCA (Set Latest DCA) and SETTRACKTOLATESTDCA (Set Track to Latest DCA) cue command and sample below. This is a wonderful way to work your cues for running a show.

**NOTE: THIS PORTION OF THE DOUCMENT REFLECTS AN EARLIER VERION OF COMMANDS.
EACH COMMAND MUST BE PRECEEDED BY “devicename|” ... example:
WING1|createfile|**

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to the existing file. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.

Format:

createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:

- 1) it calls createfile to open a new output cue file based on the name of the input cue file
- 2) it renames the matrix 8 channel to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
- 3) it sets the Delay value

Format:

CUSTOMStartCue|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:

- 1) it renames Matrix 8 Channel to the specified text (perhaps the Cue name) to indicate to you that this is completed

Format:

CUSTOMEndCue|text|

TRACKMUTE

TRACKMUTE is used to mute or unmute a track

Format:

TRACKMUTE|myTrackVar's name|MUTE or UNMUTE|

TRACKLED

TRACKLED is used to turn the track LED on or off

Format:

TRACKLED|myTrackVar's name|ON or OFF|

TRACKNAME

TRACKNAME is used to set a track name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)

Format:

TRACKNAME| myTrackVar's name |text|

TRACKCOLOR

TRACKCOLOR is used to set the color for a track in the scribble strip. The Color Names are set in the BUILD script (sample: LIGHTBLUE, MEDIUMBLUE, DARKBLUE, TURQUOISE, GREEN, OLIVEGREEN, YELLOW, ORANGE, RED, CORAL, PINK, MAUVE).

Format:

TRACKCOLOR | myTrackVar's name | Color Name |

TRACKICON

TRACKICON is used to set a track icon in the scribble

Format:

TRACKICON | myTrackVar's name | icon number |

TRACKPAN

TRACKPAN is used to set a pan a track. Values are -100 to 100

Format:

TRACKPAN | myTrackVar's name | number |

TRACKPOSTINS

TRACKPOSTINS is used to assign a post insert for a track.

Format:

TRACKPOSTINS | myTrackVar's name | FX or AUTO_X or AUTO_Y |

TRACK_GATE_ON

TRACK_GATE_ON is used turn the GATEeffect on for a track.

Format:

TRACK_GATE_ON | myTrackVar's name |

TRACK_GATE_OFF

TRACK_GATE_OFF is used turn the GATE effect off for a track.

Format:

TRACK_GATE_OFF | myTrackVar's name |

TRACK_EQ_ON

TRACK_EQ_ON is used turn the EQ effect on for a track.

Format:

TRACK_EQ_ON | myTrackVar's name |

TRACK_EQ_OFF

TRACK_EQ_OFF is used turn the EQ effect off for a track.

Format:

TRACK_EQ_OFF | myTrackVar's name |

TRACK_DYN_ON

TRACK_DYN_ON is used turn the DYN effect on for a track.

Format:

TRACK_DYN_ON | myTrackVar's name |

TRACK_DYN_OFF

TRACK_DYN_OFF is used turn the DYN effect off for a track.

Format:

TRACK_ DYN _OFF| myTrackVar's name |

TRACK_PREINS_ON

TRACK_ PREINS _ON is used turn the PREINS effect on for a track.

Format:

TRACK_ PREINS _ON| myTrackVar's name |

TRACK_PREINS_OFF

TRACK_ PREINS _OFF is used turn the PREINS effect off for a track.

Format:

TRACK_ PREINS _OFF| myTrackVar's name |

TRACK_POSTINS_ON

TRACK_ POSTINS _ON is used turn the POSTINS effect on for a track.

Format:

TRACK_ POSTINS _ON| myTrackVar's name |

TRACK_POSTINS_OFF

TRACK_ POSTINS _OFF is used turn the POSTINS effect off for a track.

Format:

TRACK_ POSTINS _OFF| myTrackVar's name |

TRACKPROC

TRACKPROC is used to assign a set the GEDI in the desired order for processing. See the OSC manual for options.

GED, GEID, GIED, IGED...

Format:

TRACKPROC| myTrackVar's name|text|

TRACKTAG

TRACKTAG is used to assign a TAG value to a track.

Format:

TRACKTAG| myTrackVar's name|myTagVar's name|

TRACKTAGMUTE

TRACKTAGMUTE is used to assign a TAG value to a track and mute or unmute the track

Format:

TRACKTAGMUTE| myTrackVar's name|myTagVar's name|MUTE or UNMUTE|

TRACKGROUPIN

TRACKGROUPIN is used to set a GROUP connection name and number. It is best to do this while the track is muted.

Format:

TRACKGROUPIN| myTrackVar's name|myCONNVar's name|

TRACKSEND

TRACKSEND is used to set the enable or disable a send from a specified track.

Format:

TRACKSEND | myTrackVar's name | mySend's name | MUTE or UNMUTE

SENDand_UNMUTE

SENDand_UNMUTE is used to update SENDS and then UNMUTE the track.

Format:

SENDand_UNMUTE | myTrackVar's name | mySend's name |

SENDand_MUTE

SENDand_MUTE is used to update SENDS and then MUTE the track.

Format:

SENDand_MUTE | myTrackVar's name | mySend's name |

USB_PLAY

USB_PLAY is used to control playing for the USB recorder

Format:

USB_PLAY | STOP or PAUSE or NEXT or PREV or PLAYFILE |

USB_PLAYFILE

USB_PLAYFILE is used to set the file name to play for when the PLAYFILE is sent with USB_PLAY

Format:

USB_PLAYFILE | full file name |

USB_RECORD

USB_RECORD is used to control recording for the USB recorder

Format:

USB_RECORD | STOP or REC or PAUSE or NEXT or NEWFILE |

LIVE_RECORDER

LIVE_RECORDER is used control the LIVE Card Recorder

Format:

LIVE_RECORDER | 1 or 2 | STOP or PLAY or PPAUSE or RECORD |

LOADSCENE

LOADSCENE is used to load a scene from its Tag Number

Format:

LOADSCENE | Scene Tag Number |

NAVSCENE

LOADSCENE is used to load a scene

Format:

NAVSCENE|GOPREV or GONEXT or GO or PREV or NEXT|

AUTOMIXXON

AUTOMIXXOFF

AUTOMIXYON

AUTOMIXYOFF

These are used to turn the specified AUTOMIX on or off.

Format:

AUTOMIXXON

Or

AUTOMIXXOFF

Or

AUTOMIXYON

Or

AUTOMXYOFF

CUSTOM_a_DCA_ONMUTED

Custom_a_DCA_ONMUTED is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to the specified text
2. It turns the Track's LED off
3. It mutes the track
4. In the current code is code to sets color YELLOW but this is presently commented out

Format:

Custom_a_DCA_ONMUTED|MyTrackVar's Name|text|

CUSTOM_a_DCA_ONUNMUTED

Custom_a_DCA_ONUNMUTED is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to the specified text
2. It turns the Track's LED on
3. It unmutes the track
4. In the current code is code to sets color YELLOW but this is presently commented out

Format:

Custom_a_DCA_ONUNMUTED|MyTrackVar's Name|text|

CUSTOM_a_DCA_OFF

Custom_a_DCA_OFF is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to "."
2. It turns the Track's LED off
3. It mutes the track
4. In the current code is code to sets color DARKBLUE but this is presently commented out

Since text is ignored you can just change the command from a text matching the Custom_a_DCA_ONMUTED or Custom_a_DCA_ONUNMUTED commands

Format:

Custom_a_DCA_OFF|MyTrackVar's Name|text{but text is ignored}|

Custom_a_CHAN_ON

Custom_a_CHAN_ON is a custom code area used to set up a channel track possibly with AUTO MIX and with a DCA assignment

1. It sets the Track's Post Insert to the specified text (like FX or AUTO_X or AUTO_Y)
2. It turns the Track's Post Insert On
3. It sets the Track's LED on
4. It turns the Track's Post Insert On
5. It sets the Track's DCA tags to match the specified DCA Variable content
6. It runs the tracks through the SENDand_UNMUTE sequence based on the Send String content
7. It colors the track as indicated in COLOR
8. It names the track as indicated in TEXT
9. It unmutes the track

Format:

Custom_a_CHAN_ON|MyTrackVar's Name|MyDCAVar's Name|post insert text|MySendString name|COLOR|TEXT|

Custom_a_CHAN_OFF

Custom_a_CHAN_OFF is a custom code area used to set up a channel track possibly with an "off" status...meaning:

1. It sets the Track's Post Insert to FX
2. It turns the Track's Post Insert On
3. It sets the Track's LED off
4. It sets the Track's DCA tags to match the "dcapark" DCA Variable content
5. It runs the tracks through the SENDand_MUTE sequence for "spark" including that it mutes the track although LED is off, COLOR is set to dark blue
6. It names the track as indicated in TEXT

The inclusion of and non-use of parameters makes it easier to keep editing cues and toggling between these 2 custom commands.

Format:

Custom_a_CHAN_OFF|MyTrackVar's Name|MyDCAVar's Name{ignored}|post insert text {ignored}|MySendString name{ignored}|COLOR{ignored}|TEXT|

SETLATESTDCA

:SETLATESTDCA sets the latest DCA to the specified DCA

Format:

SETLATESTDCA|TagVar's name or xxx for not having one|myTrackVar for the DCA|Name to set on the DCA Track

SETTRACKTOLATESTDCA

SETTRACKTOLATESTDCA sets the specified Track to have the Tag from the SETLATESTDCA value

- It sets the tags according to the TagVar in SETLATESTDCA and
- If the Tag content is xxx then it mutes the track and turns off the LED (and removes all tags)
- otherwise, it does set the tags according to the TagVar in SETLATESTDCA and turns on the LED and Unmutes the specified track

Format:

SETTRACKTOLATESTDCA|myTrackVar's name|

These 2 commands allow you to just place a SETLATESTDCA command in a CUE and then follow it by the SETTRACKTOLATESTDCA commands for all tracks you want in that DCA.

If you use xxx in the SETTRACKTOLATESTDCA name then whatever tracks that follow will be muted.

With these you can cut and paste SETTRACKTOLATESTDCA text lines under the desired DCAs to manage the cues. This is very efficient and easy to manage.

Here is sample CUE file to demonstrate this:

```
WING1|CUSTOMStartCue|"Cue 140 Running"|
REAPER1|#|ACTIONi|401574
REAPER1|#|LASTMARKERName||"Cue 140"|

WING1|#| ----- overall auto mix - everyone is in X excluding ENS-only -----

WING1|AUTOMIXXOFF|

WING1|SETLATESTDCA|w1dcaOTHER|w1dOTHER|" ariel plt grims"|
WING1|SETTRACKTOLATESTDCA | w1cCaroline | # Ariel
WING1|SETTRACKTOLATESTDCA | w1cKeith | # Grimsby
WING1|SETTRACKTOLATESTDCA | w1cSyr | # Pilot

WING1|SETLATESTDCA|w1dcaens1|w1dENS1|". "|
WING1|SETLATESTDCA|w1dcaens2|w1dENS2|". "|
WING1|SETLATESTDCA|w1dcaens3|w1dENS3|". "|
WING1|SETLATESTDCA|w1dcaens4|w1dENS4|"7sailors"|
WING1|SETTRACKTOLATESTDCA | w1cDavid | # Sailor Chef
WING1|SETTRACKTOLATESTDCA | w1cRobert | # Sailor Chef
WING1|SETTRACKTOLATESTDCA | w1cThomas | # Sailor Chef
WING1|SETTRACKTOLATESTDCA | w1cBill | # Sailor Chef
```

WING1|SETTRACKTOLATESTDCA | w1cDennis | # Sailor Chef -Chef Louis US-
WING1|SETTRACKTOLATESTDCA | w1cJoshua | # Sailor Chef -Grimsby US-
WING1|SETTRACKTOLATESTDCA | w1cRay | # Sailor Chef -Triton US-
WING1|SETTRACKTOLATESTDCA | w1cWilliam | # Gull -Eric US-

WING1|SETLATESTDCA|xxx|
WING1|SETTRACKTOLATESTDCA | w1cGabriel | # Eric
WING1|SETTRACKTOLATESTDCA | w1cChadly | # Sebastian
WING1|SETTRACKTOLATESTDCA | w1cAnna | # Flounder
WING1|SETTRACKTOLATESTDCA | w1cMargery | # Ursula
WING1|SETTRACKTOLATESTDCA | w1cJohn | # Triton
WING1|SETTRACKTOLATESTDCA | w1cJudah | # Chef Louis
WING1|SETTRACKTOLATESTDCA | w1cBridget | # Scuttle
WING1|SETTRACKTOLATESTDCA | w1cDylan | # Flotsam
WING1|SETTRACKTOLATESTDCA | w1cElijah | # Jetsam
WING1|SETTRACKTOLATESTDCA | w1cKatelyn | # Aquata Princess 1
WING1|SETTRACKTOLATESTDCA | w1cNadja | # Andrina Princess 2 Gull
WING1|SETTRACKTOLATESTDCA | w1cRachelP | # Arista Princess 3
WING1|SETTRACKTOLATESTDCA | w1cMaura | # Atina Princess 4
WING1|SETTRACKTOLATESTDCA | w1cJacklyn | # Adella Princess 5
WING1|SETTRACKTOLATESTDCA | w1cSarah | # Allana Princess 6 -Ariel US-
WING1|SETTRACKTOLATESTDCA | w1cRebecca | # Maid Gull
WING1|SETTRACKTOLATESTDCA | w1cAnnie | # Maid
WING1|SETTRACKTOLATESTDCA | w1cKara | # Maid
WING1|SETTRACKTOLATESTDCA | w1cGenevieve | # Leeward
WING1|SETTRACKTOLATESTDCA | w1cJocelyn | # Windward
WING1|SETTRACKTOLATESTDCA | w1cRachels | # -MerSister US-
WING1|SETTRACKTOLATESTDCA | w1cJen | # -Ursula US-
WING1|SETTRACKTOLATESTDCA | w1cPriscilla | # Gull
WING1|SETTRACKTOLATESTDCA | w1cOS1 | # -offstage mic 1
WING1|SETTRACKTOLATESTDCA | w1cOS2 | # -offstage mic 2
WING1|SETTRACKTOLATESTDCA | w1cOS3 | # -offstage mic 3
WING1|SETTRACKTOLATESTDCA | w1cOS4 | # -offstage mic 4

WING1|CUSTOMEndCue|"Cue 140 Done"|

#|

This is the comment indicator

comments begin with #|

Format

#|

X32 M32

CUES for X32/M32

NOTE: THIS PORTION OF THE DOUCMENT REFLECTS AN EARLIER VERION OF COMMANDS.

EACH COMMAND MUST BE PRECEDED BY "devicename|" ... example:

X1|createfile|

Each Cue file that you build contains commands that the BUILD script will read and create an expanded cue file for. Those cue files will be used by the RUN script later to run your cues.

Next is the detail on the commands that you place in a CUE file. Note that if you build these files in sequence that you limit the amount of adjustment you need to make in them and it really is pretty fast.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can used Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

```
reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

For the Current User and setting the sampleext extension to be recognized as text:

```
reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

Note that Notepad++ is freely available and can be used to make mass changes to multiple files at once if needed to adjust your cue file.

Commands in the cue files are executed in the order received, so be sure you consider that when placing the content. Things that must be done quickly should be first.

Because cues can be run out of sequence, the policy leveraged for this example is to set everything in a track than can change in any cue in every cue. When things have to be done extremely quickly and back to back, that policy can have an exception. However, in my usage I name those cues with a "Q" in the name of the cue to identify it visually as a Quick cue and not a starting point.

Note that cue names and cue sequence are not the same. The sequence of cue file execution is specified in the RUN script.

Next below are the definitions of the commands that can be placed in the cue files. This will help you see what you can designate.

To build the Cue files for the RUN script to process, just run the BUILD script from within the directory with the other items and it will build all the cue expanded files that the RUN script will process.

Detail Using an Example for X32/M32 BUILD Script Modifications

Channels

In the sample script you will see designations of channels like this:

```
set myChan.testchan1=01
set myChan.testchan2=02
set myChan.testchan3=03
```

Do not have spaces or characters that a CMD script will misinterpret in these names. Ideally stick to alphanumeric and underscore for safety.

For this example, “testchan1” is the name to identify the 1st channel in some X32 instance being controlled, etc. These are names you give to these channels for scripting. Note that the values MUST be 2 digits so include the leading 0 for Channels 0-9.

Numbers can be repeated, but names must not. Names are used across all controlled systems and each channel in those systems must be unique. It is these names that you will specify in your cues. This allows you to change things around in your system and only requires you to change the number here instead of making you edit each cue file if you move things around.

BUS IDENTIFIERS

In your cues, you will also be able to send tracks to specific BUSs.

The BUILD script is where you place your definitions, such as:

```
set myBus.testbus1=01
set myBus.testbus2=02
set myBus.testbus3=03
set myBus.testbus4=04
set myBus.testbus5=05
```

testbus1 is the name used in Cue files you write to refer to Bus 1, for example. This allows you to change bus assignments in the BUILD definitions instead of in each cue file if needed. You can refer to BUSs by these names in cues.

SEND IDENTIFIERS

In the SENDS designations note the following meaning of the SEND strings:

1 sets the send in this position to ON, or Unmuted

0 sets the send in this position to OFF, or Muted

X leaves the send alone

* terminates the processing of the send string

- is ignored and does not even count as a position

Sample from this script:

```
set mySend.solo=1000*  
set mySend.soloplusverb=0001*  
set mySend.muteall=0000-0000-0000-0000*  
set mySend.unchanged=*
```

The names specified here “solo”, etc.) are used in the cue files.

Each 1 or 0 specified what to do with the send at that sequence position.

For example solo is:

*1000**

Position 1 is a 1 so BUS 1 Send would be set ON for the specified channel when this SEND is used in a command.

The other 0's in position 2, 3,4 mean BUS 2, 3, 4 Sends for the referenced Channel will be set to OFF.

– is a spacer and does not have any meaning or cause the position to increment, it just helps make things readable in the code.

* Ends the string of entries

DCA Assign Identifiers

You name your DCAs for reference in CUES for assigning DCAs to a Channel.

For the example script you will see:

```
set myDCAassign.testassigndca1=1  
set myDCAassign.testassigndca2=5
```

BE CAREFUL—the numbers are not the DCA number, but rather are the DCA reference value the X32 uses.

Values range from 0 to 255.

- DCA 1 = 1
- DCA 2 = 2
- DCA 3 = 4
- DCA 4 = 8

- DCA 5 = 16
- DCA 6 = 32
- DCA 7 = 64
- DCA 8 = 128

This allows you to specify multiple DCAs for a single reference by adding values together.

For example, if you set testassigndca2 as a name of value 5, as above, then testassigndca2 refers to a combination of DCAs 1 and 3 since this is the of the values for DCAs 1 and 3 (1+4=5).

Note that this is used in setting a channel to be included in one or more DCAs. It is not how DCA names, colors, or mutes are handled.

DCA Identifiers

You name your DCAs for reference in CUES for DCA color, mute handling, and naming.

For the example script you will see:

```
set myDCA.testdca1=1
set myDCA.testdca2=2
```

These are names for DCAs when handling DCA attributes and range in value from 1-8. NOTE These are not used to assign channels to DCAs. See above for that.

Route Out Identifiers and Route Source Out

The x32 allows definition of 208 output choices for USER OUT. Since User Out is how you may want to designate how audio inputs are mapped to REAPER, being able to change these will be handy. If you use a DN9630, for example, you can send User Out 1-48 to an AES50 to go to the DN9630 to go to REAPER, with AES50x 1 referred to at input 1 in REAPER, etc.

While you can have 168 different audio sources connected to the x32, and a choice of sending any of 208 items as outputs, you can only send 48 at a time to this interface, for example, and can use User Out to specify them. By keeping an x32 and REAPER aligned, you can change routing this way to get up to 48 of any audio sources the x32 offers to REAPER at any one time, but you have access to all of them as choices to select from.

Jumping ahead of ourselves a bit, cues can contain a ROUTE_OUT line that looks like this:

```
ROUTE_OUT|xxx|yyy|
```

Where xxx is a Route Out Identifier you set up and yyy is a Route Name. Route Names are predefined in the BUILD script for you (you can change them). For Example RouteSourceOut "Local1" is set to a value of 1. For the x32, the value of 1 is defined as local input 1 connected to the board.

If you define testroute1 to equal 01 then place that in CUE file like this:

ROUTE_OUT|testoutroute1|LOCAL1

You would run the CUE to have the x32 set User Out 01 to be what x32 sees as the input at Local 1.

If you then map User Out 1 to AES50B 1 and connect a DN9630 to AES50 B and run REAPER as your DAW/Mixer you would get whatever device is connected to Local 1 as input 1 in REAPER. In this setup you can have up to 48 inputs flowing at any 1 time, and by changing routing in a CUE you can change which inputs those are as needed, If you manage REAPER at the same time, you would mute the track for the no longer needed device, change the routing on the x32, and unmute the REAPER track for the newly assigned device, This can all be done with these scripts and the SEND script, and you can make the CUEs automatically follow each other from one trigger cue or can run them manually,

Back to the sample provide BUILD script, you will see these as examples:

```
set myRouteOut.testoutroute1=01
set myRouteOut.testoutroute2=02
set myRouteOut.testoutroute3=03
set myRouteOut.testoutroute4=04
set myRouteOut.testoutroute5=05
```

Note that these MUST BE 2 digit values, so include the leading 0. Values range from 01 to 48.

You will also see these Route Names the give the x32 designated route number a name that matches the connection used.

```
set myRouteSourceOut.OFF=0
set myRouteSourceOut.LOCAL1=1
set myRouteSourceOut.LOCAL2=2
set myRouteSourceOut.LOCAL3=3
set myRouteSourceOut.LOCAL4=4
set myRouteSourceOut.LOCAL5=5
set myRouteSourceOut.LOCAL6=6
set myRouteSourceOut.LOCAL7=7
set myRouteSourceOut.LOCAL8=8
set myRouteSourceOut.LOCAL9=9
set myRouteSourceOut.LOCAL10=10
```

You may find renaming things to be useful, as the provided detail is just examples.

For example:

```
set myRouteOut.REAPERInput_1=01
set myRouteOut.REAPERInput_2=02
set myRouteOut.REAPERInput_3=03
set myRouteOut.REAPERInput_4=04
set myRouteOut.REAPERInput_5=05
set myRouteOut.REAPERInput_6=06
and
```

```
set myRouteSourceOut.OFF=0
```

```
set myRouteSourceOut.CELLO1=1
set myRouteSourceOut.CELLO2=2
set myRouteSourceOut.MIC1=3
set myRouteSourceOut.MIC2=4
set myRouteSourceOut.MIC3=5
```

Then you can say in a CUE:

```
ROUTE_OUT|REAPERInput_1|CELLO1
ROUTE_OUT|REAPERInput_2|CELLO2
ROUTE_OUT|REAPERInput_3|MIC1
ROUTE_OUT|REAPERInput_4|MIC2
ROUTE_OUT|REAPERInput_5|MIC3
```

This will set User Out to the specified devices.

Later, if you are not using MIC 1, you could reassign REAPER input 3 to something else. This gives you the ability to change what is sent on the fly among all connected equipment.

Be sure to keep a record of the mapping to numbers in the x32 definitions:

- 0 is OFF
- 1-32 are Local 1-32
- 33-80 are AES50 A 1-48
- 81-128 are AES50 B 1-48
- 129-160 are Card In 1-32
- 161-166 are AUX in 1-6
- 167 is TalkBack Internal
- 168 is Talkback External
- 169-184 are OUIT 1-16
- 185-200 are P16 1-16
- 201-206 are AUX 1-6
- 207 is Main L
- 208 is Main R

Route In Identifiers and Route Source In

The x32 allows definition of 168 input choices for USER IN. This is how you can designate what the x32 sees as inputs at any one time. In the example setup for the ROUTE OUT information above, if the DN9630 is connected to AES50 B then REAPER outputs 1-48 would go to AES50 B 1-48 for access at the x32. You may want to have a backup plan to pull these directly from the connections in case of REAPER failure, in which case you could have a cue prepped to revamp things. You also just may want to change inputs for other reasons or use this for setup.

Be sure to keep a record of the mapping to numbers in the x32 definitions that can be used for user in routing:

- 0 is OFF
- 1-32 are Local 1-32

- 33-80 are AES50 A 1-48
- 81-128 are AES50 B 1-48
- 129-160 are Card In 1-32
- 161-166 are AUX in 1-6
- 167 is TalkBack Internal
- 168 is Talkback External

The sample script includes the following:

```
set myRouteIn.testinroute1=01
set myRouteIn.testinroute2=02
set myRouteIn.testinroute3=03
set myRouteIn.testinroute4=04
set myRouteIn.testinroute5=05
```

Note that 2 digit values are needed and they can range from 0-32.

The myRouteSourceIn values in the BUILD script are the name references for these commands.

Colors

Colors for x32 scribble strips are passed in OSC as numbers, but designated in CUEs as names. A default set of names is provided in the BUILD script, but you can change these.

These lines define the 15 color choices:

- *set myColor.OFF=0*
- *set myColor.RD=1*
- *set myColor.GN=2*
- *set myColor.YE=3*
- *set myColor.BL=4*
- *set myColor.MG=5*
- *set myColor.CY=6*
- *set myColor.WH=7*
- *set myColor.OFFi=8*
- *set myColor.RDi=9*
- *set myColor.GNi=10*
- *set myColor.YEi=11*
- *set myColor.BLi=12*
- *set MyColor.MGi=13*
- *set myColor.CYi=14*
- *set myColor.WHi=15*

In CUEs you refer to these by OFF, RD, etc.

Card Commands

The xLive card can be manipulated in CUES. In the X32 the various actions are numbers, but they are mapped to names for CUE use.

The following are defined in the script, but you can change these

- `set myCard.STOP=0`
- `set myCard.PLAY=2`
- `set myCard.PAUSEPLAY=1`
- `set myCard.RECORD=3`

In CUEs you refer to these by STOP, PLAY, etc.

USB Stick Commands

The USB stick can be manipulated in CUES. In the X32 the various actions are numbers, but they are mapped to names for CUE use.

The following are defined in the script, but you can change these

- `set myTape.STOP=0`
- `set myTape.PLAY=2`
- `set myTape.PAUSEPLAY=1`
- `set myTape.RECORD=4`
- `set myTape.PAUSERECORD=3`

In CUEs you refer to these by STOP, PLAY, etc.

CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS X32

NOTE: Values designated as “myXxx value” are defined in the x32 BUILD script and described above.

NOTE: THIS PORTION OF THE DOUCMENT REFLECTS AN EARLIER VERION OF COMMANDS.

EACH COMMAND MUST BE PRECEDED BY “devicename|” ... example:

X1|createfile|

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to the existing file. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.

Format:

createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:

- 1) it calls createfile to open a new output cue file based on the name of the input cue file
- 2) it renames the main stereo channel to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running

3) it sets the Delay value

Format:

CUSTOMStartCue|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:

1) it renames Main Stereo Channel to the specified text (perhaps the Cue name) to indicate to you that this is completed

Format:

CUSTOMEndCue|text|

CHAN

CHAN is used to mute or unmute a channel

Format:

CHAN|myChan value|MUTE or UNMUTE

CHANNAME

CHANNAME is used to set a channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)

Format:

CHANNAME| myChan value|text

CHANCOLOR

CHANCOLOR is used to set the color for a channel in the scribble strip. The Color Names are set in the BUILD script.

Format:

CHANCOLOR| myChan value|Color Name

CHANSEND

CHANSEND is used to set the enable or disable a send from a specified channel.

Format:

CHANSEND|myChan value| myBus value |MUTE or UNMUTE

CHANDCA

CHANDCA is used to set the a Channel into one or more DCAs.

Format:

CHANDCA| myChan value|myDCAassign value|

DCA

DCA is used to mute or unmute a DCA

Format:

DCA|myDCA value|MUTE or UNMUTE

DCANAME

DCANAME is used to set a DCA name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)

Format:

DCANAME| myDCA value|text

DCACOLOR

DCACOLOR is used to set the color for a DCA in the scribble strip. The Color Names are set in the BUILD script.

Format:

DCACOLOR| myDCA value |myColor value

BUS

BUS is used to mute or unmute a BUS

Format:

BUS|myBus value|MUTE or UNMUTE

BUSNAME

BUSNAME is used to set a BUS name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)

Format:

BUSNAME| myBus value|text

BUSCOLOR

BUSCOLOR is used to set the color for a BUS in the scribble strip.

Format:

BUSCOLOR|myBus value|myColor value

MAINST

MAINST is used to mute or unmute the Main Stereo Channel

Format:

MAINST|MUTE or UNMUTE

MAINSTNAME

MAINST is used to set a MAIN Stereo Channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)

Format:

MAINSTNAME|text

MAINSTCOLOR

MAINSTCOLOR is used to set the color for the Main Stereo channel in the scribble strip.

Format:

MAINSTCOLOR|myColor value

MAINM

MAINM is used to mute or unmute the Main Mono Channel

Format:

MAINM|MUTE or UNMUTE

MAINMNAME

MAINM is used to set a MAIN Mono Channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)

Format:

MAINMNAME|text

MAINMCOLOR

MAINMCOLOR is used to set the color for the Main Mono channel in the scribble strip.

Format:

MAINMCOLOR|myColor value

SENDand_UNMUTE

SENDand_UNMUTE is used to update BUS SENDS and then UNMUTE the channel.

Format:

SENDand_UNMUTE|myChan value|mySend value|

SENDand_MUTE

SENDand_MUTE is used to mute the channel and then update BUS SENDS

Format:

SENDand_MUTE| myChan value|mySend value|

ROUTE_OUT

ROUTE_OUT is used to define a source and target for USER OUT

Format:

ROUTE_OUT|myRouteOut value|myRouteSourceOut value

ROUTE_IN

ROUTE_IN is used to define a source and target for USER IN

Format:

ROUTE_IN| myRouteIn value|myRouteSourceIn value

USB_RECORDER

USB_RECORDER is used control play and record for the USB recorder

Format:

USB_RECORDER|myTape value|

LIVE_RECORDER

LIVE_RECORDER is used control play and record for the LIVE SD recorder

Format:

LIVE_RECORDER|myCard value|

SAVESCENE

SAVESCENE is used to save a scene in slot 0-99 and name it and set a scene note. Enclose strings with spaces in them in quotes.

Format:

SAVESCENE|mySceneNumber|Name|Note|

LOADSCENE

LOADSCENE is used to load a scene from slot 0-99.

Format:

LOADSCENE|mySceneNumber|

SAVESNIPPET

SAVESNIPPET is used to save a snippet in slot 0-99 and name it. Enclose strings with spaces in them in quotes. Note that the filters used are those already assigned to this snippet slot. Therefore, you can use this to replace content of a snippet but not save a brand new snippet.

Format:

SAVESNIPPET|mySnippetNumber|Name

LOADSNIPPET

LOADSNIPPET is used to load a snippet from slot 0-99.

Format:

LOADSNIPPET|mySnippetNumber|

#|

This is the comment indicator
comments begin with #|

Format

#|

REAPER

Sample for REAPER BUILD SCRIPT MODIFICATIONS

An example will really help in understanding this, so an example is provided here. The syntax of the commands will be shown in detail later in this document.

The example here uses the input file content in the syntax for REAPER.

Using an editor (Notepad for example) you will edit the template BUILD script to define things for your usage.

The template BUILD script contains comments to help you in making these updates.

Build Script Edits

Track (Channel) Naming

Next, you need to let the system know the names you will use in your INPUT cues for each REAPER track you will reference. The Template provided assumes the custom REAPER layout will be used for the mixer I have configured. You can use the Cue system with any REAPER configuration you want though.

In the template I define the start of each section of tracks by a variable name so I can set up names relative to that offset. You do not need to do this. You can just use track numbers directly if you desire. However, every track you want to process commands against needs one or more names that you will use in your scripts.

For simplicity, I will provide an example that is not part of the template but is just illustrative. Let's say that the first track in REAPER is going to be used by an actor named Joe and Joe is playing the role of TheMan. In my scripts I would like to refer to Track 1 by 3 names: track1, joe, and theman.

In the BUILD script I would add these lines in the definition area:

```
set /a myChan.track1=1  
set /a myChan.joe=1  
set /a myChan.theman=1
```

Now, anywhere I use a reference to “track1” or “joe” or “theman” in the context of a channel or track, in an input cue I write the BUILD script will replace these names with a reference to Track 1. By using names in the cues, if they are moved to other tracks you only have to change the BUILD file and rebuild the output cues. The cues do not need editing. These Names must not contain characters that a command script will misinterpret, for safety stick to letters and numbers. Other characters may be ok, but I have not verified which are and are not valid.

Let’s say that this is the only track I want cues to operate against, so I am done defining reference names.

Send Identifiers (tracks to bus tracks sending, monitor sending, etc. based on your configuration)

In your cues, you will also be able to send tracks to specific sends. In this example, certain people have consistent sends. If you do not have dynamic sending and just have static routing you will not need to modify, you do not need to use any send identifiers.

For example, for a choir concert each person could be assigned to a SEND for a Soprano, Alto, Tenor or Bass group or a Solo track. In that scenario these 5 options are the only options a person has.

In setting up the tracks that will use these particular send identifiers, each track needs the same numbers of and positions of sends so that the offset in each does the same thing (send 1 is always to the Soprano Group, send 2 is always to the Alto group, etc. for every track for which these identifiers will be used). You can use different send identifiers for tracks that have a different set of sends configured in the system. If this is an x32, there are 16 sends for every track, those are your 16 buses. In REAPER this can be anything you configure.

In the SENDS Identifier designations note the following meaning of the SEND identifier content:

1 sets the send in this position to the volume level specified (0.0 through 1.0)

0 sets the send in this position to 0

X leaves the send alone

* terminates the processing of the send identifier

- is ignored and does not even count as a position in the send identifier, it is just used to make the strings easier to read.

For example:

```
set mySend.soprano=1000-0*
set mySend.alto=0100-0*
set mySend.tenor=0010-0*
set mySend.bass=0001-0*
set mySend.solo=0000-1*
set mySend.muteall=0000-0*
set mySend.unchanged=*
```

The names specified here “solo”, “soprano”, etc.) are used in the cue files in various commands.

For example an input cue file you write could contain:

SENDand_UNMUTE |joe |soprano |.6

This would, using this example, for the track “joe” references set the first send to volume level 0.6, the 2nd send for that track to volume 0, same for the 3rd, 4th, and 5th sends. It would then Unmute the “joe” track.

To avoid confusion, I recommend starting all send identifier names with an “s” to help ensure you know what is going on contextually.

Plugin Adjust identifiers (REAPER only, experimental)

In your cues, you will also be able to adjust an fx parameter using the same type of identifier as used for Sends, but with a different naming convention.

You will have to edit the BUILD script code to set the fx Parameter number you want to adjust this way. You can also duplicate the code or write a different command if you would like based on the example code. I do not yet have a use for this, so consider it experimental.

In the Plugin Identifier designations note the following meaning of the identifier content:

{anything else} runs whatever code you placed in the processing portion of the BUILD script for a value of whatever single character you placed in this position (you need to edit the code to have this do what you want)

x does nothing

* terminates the processing of the plugin adjust identifier

- is ignored and does not even count as a position , it is just used to make the strings easier to read.

For example:

```
set myPlugin.dothis=abc*
```

This will execute whatever you coded to happen when an “a” appears to the first plugin and will also do whatever you coded when a “b” appears to the second plugin of the specified track, and whatever you coded when a “c” appears to the 3rd plugin.

The default code only looks for “a”. The command format is PLUGIN_ADJ|track|pluginadjidentifier| but you could add 2 more values if coded for.

When an “a” appears in position “n”, in the sample code included in the script, the code will write the OSC to the output file to set the value of the 1st parameter of the nth plugin of the specified track to 1.

To avoid confusion, I recommend starting all send identifier names with a “p” to help ensure you know what is going on contextually.

Sample of Cue File Content (REAPER syntax) for reference

NOTE: THIS PORTION OF THE DOUCMENT REFLECTS AN EARLIER VERION OF COMMANDS. EACH COMMAND MUST BE PRECEEDED BY “devicename|” ... example:
REAPER1|createfile|

I can now write some cues. Let’s say I want to mute Track 1 and Arm Track 1 and Name Track 1 to Joe. In Notepad, create a file named A_Setup.oz Recall that the first letter “A” will be tied to the IP and Port to send the Cue file to and the file extension oz indicates that this is part of project oz. A project can contain one or more file prefixes since a project may require sending OSC commands to one or more devices.

In the cue file place these “commands:”

```
createfile
MUTE|joe|
TRACKNAME|joe|Joe|
ARMON|joe|
```

Let’s add another cue file to unmute this track. In Notepad create a file named A_Start.oz with this content:

```
createfile
UNMUTE| |joe|
```

Great, we now have 2 cue files. The “createfile” command tells the RUN program to start a new Cue file of the proper name. Without this the new content written would be appended to the prior file by the same name. That is generally not a desired way to build cue files.

CUES for REAPER

Each Cue file that you build contains commands that the BUILD script will read and create an expanded cue file for. Those cue files will be used by the RUN script later to run your cues.

Next is the detail on the commands that you place in a CUE file. Note that if you build these files in sequence that you limit the amount of adjustment you need to make in them and it really is pretty fast.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can used Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

```
reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

For the Current User and setting the sampleext extension to be recognized as text:

```
reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

Note that Notepad++ is freely available and can be used to make mass changes to multiple files at once if needed to adjust your cue file.

**NOTE: THIS PORTION OF THE DOUCMENT REFLECTS AN EARLIER VERION OF COMMANDS. EACH COMMAND MUST BE PRECEEDED BY “devicename|” ... example:
REAPER1|createfile|**

Commands in the cue files are executed in the order received, so be sure you consider that when placing the content. Things that must be done quickly should be first.

Because cues can be run out of sequence, the policy leveraged for this example is to set everything in a track that can change in any cue in every cue. When things have to be done extremely quickly and back to back, that policy can have an exception. However, in my usage I name those cues with a “Q” in the name of the cue to identify it visually as a Quick cue and not a starting point.

Note that cue names and cue sequence are not the same. The sequence of cue file execution is specified in the RUN script.

Next below are the definitions of the commands that can be placed in the cue files. This will help you see what you can designate.

To build the Cue files for the RUN script to process, just run the BUILD script from within the directory with the other items and it will build all the cue expanded files that the RUN script will process.

[One Example Cue File for reference](#)

This is a cue from a run of Wizard of Oz, Jr. for an example to see. Other sample cues are provided with your download of this solution.

First, I run “createfile” to cause the BUILD script to create a new file instead of append the content to any existing output file that may already exist with this same name. The CUSTOMStartCue command could be used instead and it has added benefits, like creating a marker I the timeline at the time at which the cue was executed in the transport in REAPER. However, I did not want this to take place in this example so did not use that command.

I have a REAPPER track designated for cue information. That track has a defined script reference name of cstatusofcue. I set that to a string of text at the start and the end of the cue. In the provided track REAPER mixer layout, this appears in a text area of the screen to keep it available while mixing. In REAPER it is placed as the track name.

Then I have a comment line that starts with #|

The Dorothy character never moves from the bdorothy track, which is a bus track in REAPER. However, I set the bus name to “.” when Dorothy is muted to avoid confusion (I mix at the bus level in the layout). I include the actual person’s initials in the bus name so I can quickly find the track associated with the input for this character.

I then turn on AUTOMIX for this bus (this requires an automix plugin to be present on the bus. This is specific to use of a specific plugin for this purpose.

Next for the JohannaB track (who plays Dorothy) I set the send specified positionally by the sdorothy send identifier to volume 0.6 (which is 0 db in my scale) and unmute the track. Dorothy is then ready to be heard!

I have similarly dedicated buses for the Scarecrow/Hunk, Tin Man/Hickory, Lion/Zeke, Wicked Witch/Gulch. In this cue they are all muted. I mute them at the end of the script so that my unmutes take priority. Every Cue follows this same layout to make maintenance easy.

Next I can assign one or more source tracks to any of 5 line buses. In this example, Henry is unmuted for the Lines2 bus that is dynamically named HENRY_bg for this cue. The BeatriceG source track is routed to the blines2 bus.

I then place everyone who is muted into the “stash” area at the bottom of the cue.

You may find it add that I have a volume level and send string there. Once you see the options available for commands, you may ask why I didn’t just use the MUTE command. The reason is that the SEND_ezMUTE command ignores everything in the command except the track name to mute. It does just mute the track. However, this command lets me keep that ignored content in place and I can just cut and paste it into position for whatever I am going to use it for. Then I just select “ezMUTE” with a double-click and paste or type in over it “UNMUTE” and set the send identifier and I am ready to go and have an organized easy to maintain layout consistently between cue files.

At the end of the cue file I turn the orchestra track on (in this case it is a stereo track from a recording) and rename the track so I can know what is happening from my mixing screen.

To start the next cue, just start with this one and save it under a different name and make adjustments.

NOTE: THIS PORTION OF THE DOUCMENT REFLECTS AN EARLIER VERION OF COMMANDS. NOW EACH COMMAND MUST BE PRECEDED BY “devicename|” ... example:
REAPER1|createfile|

```
createfile
TRACKNAME|cstatusofcue|"Cue095-01 running"|
#| -----DOROTHY-----
TRACKNAME|bdorothy |DOROTHY_jb|
AUTOMIXON|bdorothy|
SENDand_UNMUTE |JohannaB |sdorothy|.6

#| -----SCARECROW HUNK-----
TRACKNAME|bscarecrow|. |
AUTOMIXON|bscarecrow|

#| -----TIN HICKORY-----
TRACKNAME|btin|. |
AUTOMIXON|btin|

#| -----LION ZEKE-----
TRACKNAME|blion|. |
AUTOMIXON|blion|
```

```

#| -----WICKED GULTCH-----
TRACKNAME|bwicked|.|
AUTOMIXON|bwicked|

#| -----
#| -----LINE1-----
TRACKNAME|blines1|EM_zc|
AUTOMIXON|blines1|

#| -----LINE2-----
TRACKNAME|blines2|HENRY_bg|
AUTOMIXON|blines2|
SENDand_UNMUTE|BeatriceG|slines2|.6

#| -----LINE3-----
TRACKNAME|blines3|.|
AUTOMIXON|blines3|

#| -----LINE4-----
TRACKNAME|blines4|.|
AUTOMIXON|blines4|

#| -----LINE5-----
TRACKNAME|blines5|.|
AUTOMIXON|blines5|

#| -----GROUP1-----
TRACKNAME|bgroup1|-|
AUTOMIXON|bgroup1|

#| -----GROUP2-----
TRACKNAME|bgroup2|-|
AUTOMIXON|bgroup2|

#| -----STASHED-----
SENDand_ezMUTE|AvaE|slion|.6
SENDand_ezMUTE|ZoeyC|slines1|.6
SENDand_ezMUTE|AlexandriaE|stin|.6
SENDand_ezMUTE|ShayleeW|sscawcrow|.6
SENDand_ezMUTE|SamiM|slines3|.6
SENDand_ezMUTE|LeightonP|slines1|.6
SENDand_ezMUTE|FelixH|slines1|.6
SENDand_ezMUTE|KatieB|swicked|.6
SENDand_ezMUTE|JalonK|slines2|.6
SENDand_ezMUTE|DevynnY|slines1|.6
SENDand_ezMUTE|OliviaB|sgroup1|.6
SENDand_ezMUTE|EliaD|sgroup1|.6
SENDand_ezMUTE|AlisonK|sgroup1|.6
SENDand_ezMUTE|ChloeH|sgroup1|.6
SENDand_ezMUTE|AnabellaS|sgroup1|.6
SENDand_ezMUTE|BriaR|sgroup1|.6
SENDand_ezMUTE|LiliaC|sgroup1|.6

```

```
#| -----ORCHESTRA-----
TRACKNAME|borch|ORCH|
AUTOMIXOFF|borch|
SENDand_UNMUTE |Orch    |sorch    |.6
```

```
TRACKNAME|cstatusofcue|"Cue095-01 completed-advance after Henry says OF COURSE WE BELIEVE YOU DOROTHY"
```

REAPER Cue Content-Command Syntax and Definitions

Below is information on the commands that can be placed in the cue files you write. The BUILD script recognizes these and uses them to create similarly named output files that the RUN script will execute when you use the RUN script.

- NOTES: Values designated as “myXxx value” are defined in the BUILD script as described earlier in this document.
- Commands are case-sensitive.
- Text values being passed must be in double quotes if they contain spaces.
- “Channel” and “Track” are used to mean the same thing.
- **NOTE: EACH COMMAND MUST BE PRECEDED BY “devicename|” ... example:**
- **REAPER1|createfile|**

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to an existing file by the same name if it exists. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.

Format:

```
createfile|
```

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:

- 1) it open a new output cue file based on the name of the input cue file
- 2) it renames the requested track to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
- 3) it sets the Delay value based on the value specified in the script.

Format:

```
CUSTOMStartCue|myChan value|text|
```

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:

- 1) it renames the requested track to the specified text (perhaps the Cue name) to indicate to you that this is completed
- 2) it sets up a REAPER action call to 40157 to create a marker on the transport timeline
- 3) It names that marker by the text specified

Format:

CUSTOMEndCue| myChan value |text|

SENDand_UNMUTE

SENDand_UNMUTE is used to update SENDS to a volume specified by number and then UNMUTE the channel.

Format:

SENDand_UNMUTE|myChan value|mySend value |number |

SENDand_MUTE

SENDand_MUTE is used to mute the channel and then update SENDS to a volume specified by the number

Format:

SENDand_MUTE|myChan value|mySend value|number|

SENDand_ezMUTE

SENDand_ezMUTE is used to mute the channel. The remaining fields are ignored. This command exists so that you can copy and paste commands in cue files and undertake this simple mute without having to change the format of the command. For example, if you are using SENDand_UNMUTE to set sends and then unmute a track, you can just double click on UNMUTE and replace the text with ezMUTE. The track can be unmuted again by just going back and not having to retype the rest of the command.

Format:

SENDand_ezMUTE|myChan value|

SENDand_UNMUTE_NAME

SENDand_UNMUTE_NAME is used to update SENDS and then UNMUTE the channel and set the channel's display to text and volume level to number.

Format:

SENDand_UNMUTE_NAME|myChan value|mySend value|text|number|

SENDand_MUTE_NAME

SENDand_MUTE_NAME is used to mute the channel and then update SENDS to volume in number and set the channel's display name to text.

Format:

SENDand_MUTE_NAME|myChan value|mySend value|text|number|

TRACKPAN

TRACKPAN is used to pan the track. 0 is full left, 1 if full right, .5 is center.

Format:

TRACKPAN|myChan value|pan|

SELECT

SELECT is used to select a track. The Name is optional and if specified renames the track as well as selects it.

Format:

SELECT|myChan value|Name|

UNSELECT

UNSELECT is used to un-select a track. The Name is optional and if specified renames the track as well as unselects it.

Format:

UNSELECT|myChan value|Name|

UNMUTE

UNMUTE is used to UNMUTE the channel and set the channel's display name to text.

Format:

UNMUTE|myChan value|text|

MUTE

MUTE is used to MUTE the channel and set the channel's display name to text.

Format:

MUTE|myChan value|text|

MARKERName

MARKERName to update the name of the specified marker.

Format:

MARKERName| |ID|Name|

LASTMARKERName

LASTMARKERName to update the name of the last marker.

Format:

LASTMARKERName| |Name|

GOTOMarker

GOTOMarker is used to go to the nth marker (not Marker number "N" but positionally Nth)

Format:

GOTOMarker| |number|

RECORD

RECORD is used to toggle start of transport.

Format:

RECORD

PLAY

PLAY is used to toggle play of transport.

Format:

PLAY

STOP

STOP is used to toggle stop of transport.
STOP

PAUSE

PAUSE is used to toggle pause of transport
Format:
PAUSE

FXBYPASS

FXBYPASS is used to bypass an FX for a track
Format:
FXBYPASS | my Chan value | fx number |

FXACTIVE

FXACTIVE is used to activate an FX for a track
Format:
FXACTIVE | my Chan value | fx number |

TRACKNAME

TRACKNAME is used to change the display name of a track
Format:
TRACKNAME | my Chan's Name | new display name |

TRACKVOLUME

TRACKVOLUME is used to change the volume of a track to number
Format:
TRACKVOLUME | my Chan value | number |

ACTIONi

ACTIONi takes an action number as an argument and sends it
A handy action number to keep in mind is 41743 which sends a refresh to subscribed interfacing systems. This does nothing for these scripts since they are listen only, but will be useful to refresh other devices such as TOUCHOSC interfaces. An action list can be found at <https://forum.cockos.com/showthread.php?t=23236>
Format:
ACTIONi | action number |

ACTIONS

ACTIONS takes an action text string as an argument and sends it
An action list can be found at <https://forum.cockos.com/showthread.php?t=23236>
Format:
ACTIONS | action number |

ARMON

ARMON is used to ARM the channel and set the channel's display name

ARMON|myChan's Name|Channel Display Name|

ARMOFF

ARMOFF is used to UN-ARM the channel and set the channel's display name

ARMOFF|myChan's Name|Channel Display Name|

AUTOMIXON

AUTOMIXON is a special case command. It is written to work with WT Automixer which allows up to 16 occurrences for auto mixing. What it actually does is sets fx n parameter 7 to 1 for the track specified. To work, WT Automix must be the nth fx in the track's chain. Edit the build code to change which fx is impacted.

AUTOMIXON|myChan's Name|

AUTOMIXOFF

AUTOMIXOFF is a special case command. It is written to work with WT Automixer which allows up to 16 occurrences for auto mixing. What it actually does is sets fx n parameter 7 to 0 for the track specified. To work, WT Automix must be the nth fx in the track's chain. Edit the build code to change which fx is impacted.

AUTOMIXON|myChan's Name|

PLUGIN_ADJ

PLUGIN_ADJ updates plugin settings based on a myPlugin pattern string and custom code (see details above).

PLUGIN_ADJ|myChan's Name|myPlugin's Name|optional Variable Value|optional Variable Value|

FXOPENUI

FXOPENUI is used to open an FX for a track

Format:

FXOPENUI|my Chan value|fx number|

FXCLOSEUI

FXCLOSEUI is used to close an FX for a track

Format:

FXCLOSEUI|my Chan value|fx number|

#|

This is the comment indicator

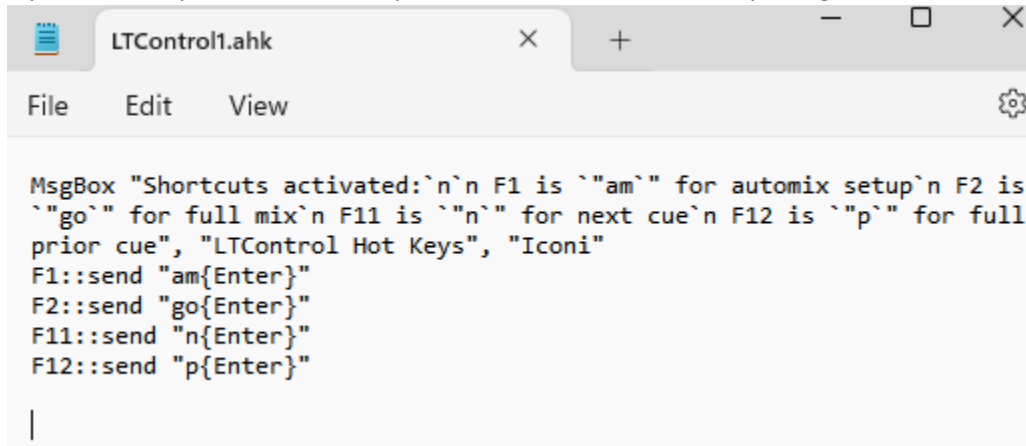
comments begin with #|

Format

#|

AutoHotKey Script

AutoHotKey Script If you want hotkeys to run utility cues in this cue system, it is recommended that you try AutoHotKey software. A sample AHT file is included in this package.



```
MsgBox "Shortcuts activated:`n`n F1 is `\"am`\" for automix setup`n F2 is `\"go`\" for full mix`n F11 is `\"n`\" for next cue`n F12 is `\"p`\" for full prior cue", "LTControl Hot Keys", "Icon1"
F1::send "am{Enter}"
F2::send "go{Enter}"
F11::send "n{Enter}"
F12::send "p{Enter}"
|
```

Figure 1 AutoHotKey sample script

Bitfocus Companion and Vireo Listener

Bitfocus Companion and the Vireo Listener can be used to provide a web interface to control the Cues. Run the Companion link with /tablet to see the simulated buttons.

Links

Download of **this document**, the **Track Muter**, **Track Muter and Volume setting** Web interface for REAPER, **Cue System** and **OSC command system**: <https://workwebs.com> or from GITHUB:

<https://github.com/ltjost/WorkWebs-Audio-Mixing-Ecosystem-Tools>

Facebook page for the Ecosystem in general and the WorkWebs tools:

<https://www.facebook.com/profile.php?id=61557113121231>

TheatreMix by James Holt: <https://theatremix.com/>

Open Stage Control OSC interface builder/tool: <https://openstagecontrol.ammd.net/>

Fully Kiosk Browser for Android: <https://www.fully-kiosk.com/>

Paul Vannatto's OSC file processing tool: <https://sourceforge.net/projects/lt-command/>

SENDOSC GitHub Link: <https://github.com/yoggy/sendosc>

AutoHotKey software for Windows: <https://www.autohotkey.com/>

SSL Solid State Logic plugin link: <https://solidstatelogic.com/products/ssl-complete-bundle-subscription>

WT Automixer for 16 tracks of Automixing in REAPER is available at: <https://www.wtautomixer.com/>

SSL 360 is available from the Support->Controllers link: <https://support.solidstatelogic.com/hc/en-gb>

REAPER: <https://www.reaper.fm/download.php>

DN9630 information: <https://klarktechnik.com/product.html?modelCode=0606-ACT>

General Behringer x32 information: <https://www.behringer.com/catalog.html>

Notepad++ powerful text editor for use on Windows: <https://notepad-plus-plus.org/>

Mixing Station: <https://mixingstation.app/>

Vicreo Listener: <https://www.vicreo-listener.com/>

Bitfocus Companion: <https://bitfocus.io/companion>

WING wosc and link to current OSC command list: <https://sites.google.com/site/patrickmaillot/wing>

VBAudio Matrix Coconut: To combine multiple ASIO compatible (and other) inputs together into a virtual interface under Windows: <https://vb-audio.com/Matrix/coconut.htm>